Lecture 14 - Vector Dialect and Tensorization

AI / NPU Compiler Course | vector.contract -> native NPU matmul

Lecture 14: Vector Dialect and NPU Tensorization Pipeline

핵심 IR boundary

vector.contract는 아직 target-neutral입니다.
NPU backend는 이를 native tensor instruction 또는 custom npu_kernel.matmul op로 합법화합니다.

NPU architect 관점

Vector shape = hardware tile contract 후보
예: vector<16x32xi8> x vector<32x16xi8> -> vector<16x16xi32>

오늘의 위치

12-13 강의 memory/buffer contract 이후 , compute tile contract 를 정의합니다 .

Lecture 14: Vector Dialect and NPU Tensorization Pipeline

핵심 IR boundary

vector.contract는 아직 target-neutral입니다.
NPU backend는 이를 native tensor instruction 또는 custom npu_kernel.matmul op로 합법화합니다.

NPU architect 관점

Vector shape = hardware tile contract 후보
예: vector<16x32xi8> x vector<32x16xi8> -> vector<16x16xi32>

Learning Objectives

- Vector Dialect 를 tensor/linalg 와 NPU kernel dialect 사이의 tile compute IR 로 이해
- vector.transfer_read/write 의 slice, permutation, mask, padding 의미 해석
- vector.contract 의 indexing_maps 와 iterator_types 를 MatMul M/N/K 로 mapping
- native NPU tile shape 에 맞는 tensorization legality checklist 작성
- positive/negative FileCheck regression test 설계

왜 NPU Compiler 에서 중요한가 ?

- Linalg 는 algorithmic structure 를 보존하지만 hardware tile instruction 은 직접 표현하지 않음
- Vector Dialect 는 n-D tile shape 와 contraction semantics 를 IR 에 명시적으로 남김
- NPU tensorization 은 vector.contract 를 target-specific matmul/tensor op 로 바꾸는 핵심 단계
  - 잘못 tensorize 하면 layout, edge tile, accumulator correctness bug 가 발생

Tensor / Vector / MemRef 비교

Tensor
- value semantic
- fusion/tiling 에 유리
- physical memory 결정 전
  - 예 : tensor<MxNxf32>

Vector
- SSA vector value
- tile compute semantics
- contract/transfer/mask
  - 예 : vector<16x32xi8>

MemRef
- storage contract
- layout/memory space
- DMA 와 runtime ABI
  - 예 : memref<..., #sram>

Vector Type Anatomy

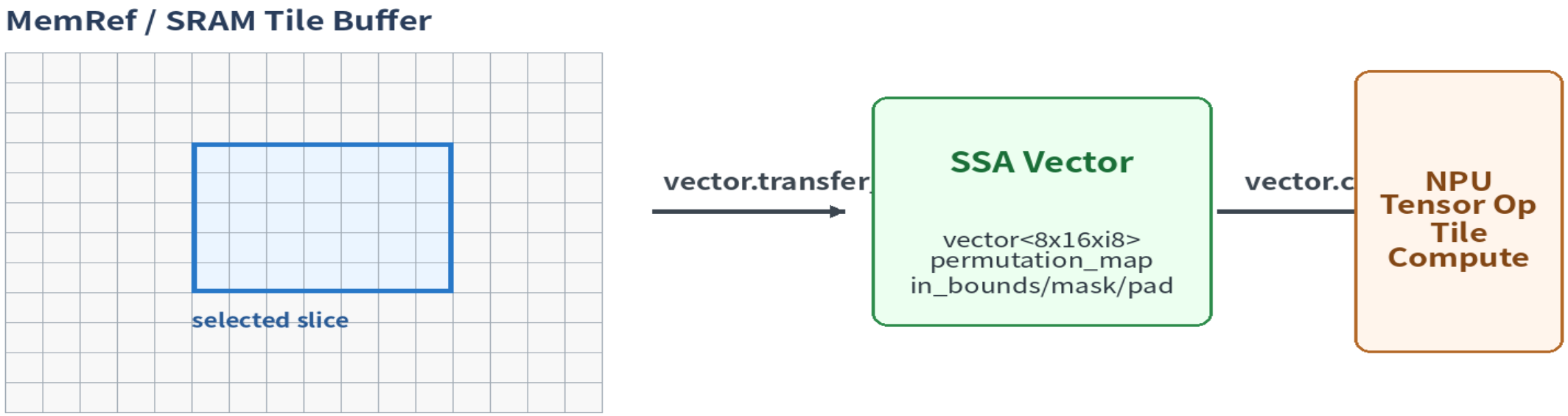
- vector<16x32xi8>: rank-2 vector, shape [16, 32], element type i8
- shape 만으로 M/K 의미가 결정되지는 않음 : vector.contract maps 와 함께 해석
- higher-dimensional vector 는 hardware native size 로 unroll/tensorize 될 수 있는 virtual tile
- NPU target description 은 legal vector shapes 를 compiler 에 제공해야 함

Core Vector Ops for NPU

- vector.transfer_read: memref/tensor slice 를 SSA vector tile 로 읽음
- vector.transfer_write: SSA vector tile 을 memref/tensor slice 로 기록
- vector.contract: matmul/dot/convolution-like contraction 표현
- vector.transpose / shape_cast: layout repair 또는 cost source
- vector.mask / create_mask: edge tile predication 표현
- vector.multi_reduction: LayerNorm, reduction kernel 후보

transfer_read/write: tile movement

vector.transfer_read / transfer_write as Tile Movement



write-back path: vector.transfer_write %Ctile, %C[%m,%n] : vector<16x16xi32>, memref<?x?xi32, #dram>

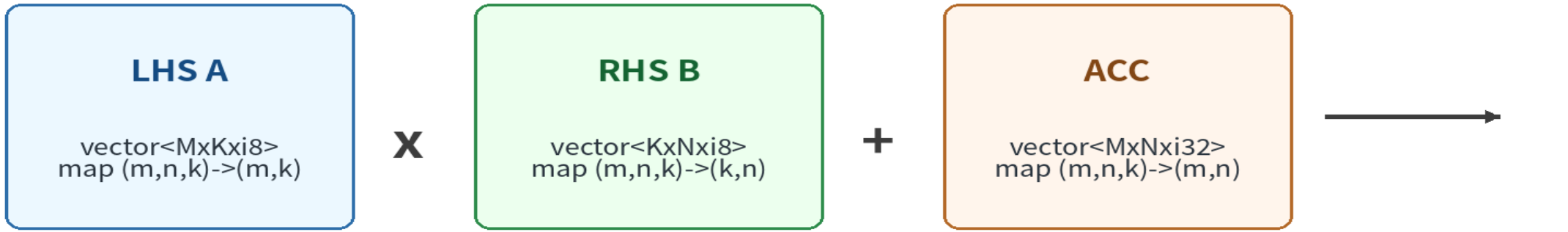
중요: transfer op는 load/store보다 큰 tile-level abstraction입니다. boundary mask, padding, permutation을 audit해야 합니다.

transfer_read Audit Checklist

- base memref/tensor: DRAM/SRAM/ACCUM memory space 확인
- indices: tile origin 과 loop IV mapping 확인
- vector type: native tile shape 와 일치하는지 확인
- permutation_map: layout transpose/repack 필요 여부 확인
- padding/mask/in_bounds: edge tile correctness 확인

vector.contract: MatMul form

vector.contract Anatomy: MatMul Form




```
#trait = {
  indexing_maps = [A(m,k), B(k,n), C(m,n)],
  iterator_types = ["parallel", "parallel", "reduction"]
}
%C = vector.contract #trait %A, %B, %Acc
    : vector<MxKxi8>, vector<KxNxi8> into vector<MxNxi32>
```

NPU tensorization은 이 contract가 native tile shape, dtype, layout, accumulator 정책과 일치하는지 검증합니다.

MatMul Contract MLIR

```
                                    #matmul_trait = {
  indexing_maps = [
    affine_map<(m, n, k) -> (m, k)>,
    affine_map<(m, n, k) -> (k, n)>,
    affine_map<(m, n, k) -> (m, n)>
  ],
  iterator_types = ["parallel", "parallel", "reduction"]
}
%c = vector.contract #matmul_trait %a, %b, %acc
  : vector<16x32xi8>, vector<32x16xi8> into vector<16x16xi32>
```

Indexing Maps 읽는 법

- iterator domain: (m, n, k)
- lhs map (m,k): A tile 은 MxK layout
- rhs map (k,n): B tile 은 KxN layout
- acc map (m,n): output tile 은 MxN layout
- iterator_types 에서 k 만 reduction 이면 MatMul contraction 후보

Accumulator 와 Mixed Types

- INT8 x INT8 는 보통 INT32 accumulator 가 필요
- BF16/FP16 은 target policy 에 따라 F32 또는 FP32-like accumulator 사용
- result type 과 accumulator type 이 항상 같은지 target ABI 에서 명시
- requant, clamp, activation 은 epilogue fusion contract 로 분리 가능
- fastmath / combining kind 는 floating-point correctness 와 연결

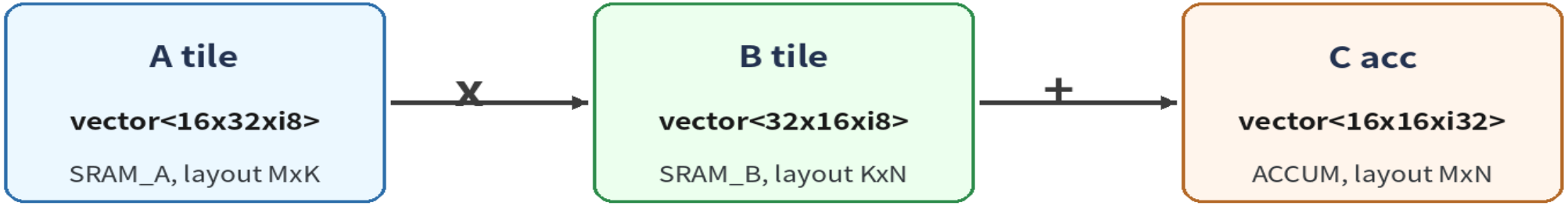
Vectorization vs Tensorization

- Vectorization: Linalg/Tensor op 를 vector.transfer + vector.contract 로 표현
- Tensorization: vector.contract 를 hardware native tensor op 로 치환
- Vector lowering: vector op 를 lower-rank vector, SCF, LLVM, target SIMD 로 낮춤
- NPU 에서는 tensorization 이 성능 critical path 이며 fallback path 와 분리해야 함

Native NPU Tile Shape

Mapping vector.contract to Native NPU MatMul

Example native tile: M=16, N=16, K=32, A/B=int8, Acc=int32, epilogue=optional requant/activation




```
npu_kernel.matmul {m=16, n=16, k=32, lhs_layout="mk",
rhs_layout="kn", acc_type="i32"}
```

Tensorization Legality: Shape

- lhs shape must encode MxK
- rhs shape must encode KxN
- acc/result shape must encode MxN
- K granularity must match dot-product datapath width
- unsupported larger K can be split/unrolled into multiple native ops

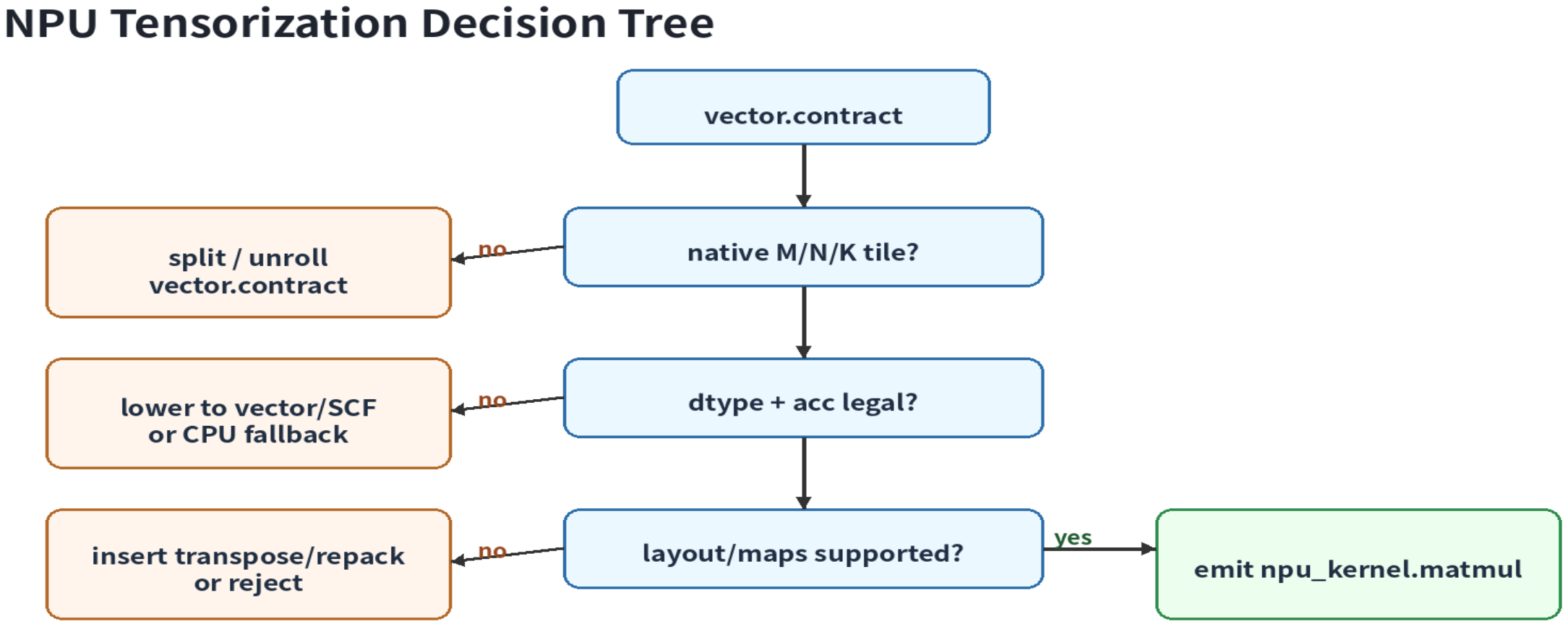
Tensorization Legality: Layout

- A(m,k), B(k,n), C(m,n) 가 native datapath 와 맞는지 확인
- B(n,k) 처럼 transposed input 이면 transpose/repack 또는 alternative op 필요
- permutation_map 과 vector.transpose 는 “무료”가 아님
- layout propagation 단계에서 tensorization-friendly layout 을 미리 유도

Tensorization Legality: Mask / Remainder

- full tile only NPU: edge tile 은 loop peeling 또는 padded copy 로 처리
- predicated NPU: vector.mask/create_mask 를 hardware predicate 로 lowering
- padding value 가 arithmetic identity 인지 확인 : matmul input pad 는 0 이어야 함
- mask 처리 정책은 correctness test 와 performance model 에 모두 반영

Tensorization Decision Tree



Rule of thumb: tensorize only when the IR expresses the hardware contract explicitly enough to prove correctness.

Case Study: INT8 GEMM Tile

- Target: M16N16K32, lhs/rhs=i8, acc=i32
- A: vector<16x32xi8>, B: vector<32x16xi8>, Acc: vector<16x16xi32>
- NPU op attrs: m=16, n=16, k=32, lhs_layout=mk, rhs_layout=kn
- Post-op: optional requant to i8/u8 and activation clamp

Case Study: BF16 MatMul Tile

- Target example: M16N16K16, lhs/rhs=bf16, acc=f32
- More bandwidth than INT8 but lower quantization complexity
- accumulator footprint can dominate SRAM/ACCUM register pressure
- fastmath flags and reproducibility requirements must be explicit

From Linalg to Vector

```
                                          // Conceptual flow
linalg.matmul ins(%A, %B : tensor<?x?xi8>, tensor<?x?xi8>)
              outs(%C : tensor<?x?xi32>)

// tile to vector size + vectorize
%a = vector.transfer_read %A[%m, %k], %c0 : tensor<?x?xi8>, vector<16x32xi8>
%b = vector.transfer_read %B[%k, %n], %c0 : tensor<?x?xi8>, vector<32x16xi8>
%out = vector.contract #matmul_trait %a, %b, %acc
  : vector<16x32xi8>, vector<32x16xi8> into vector<16x16xi32>
```

From Vector to npu_kernel

```
                                                // Before
%out = vector.contract #matmul_trait %a, %b, %acc
  : vector<16x32xi8>, vector<32x16xi8> into vector<16x16xi32>

// After
%out2 = npu_kernel.matmul %a, %b, %acc
  {m = 16 : i64, n = 16 : i64, k = 32 : i64,
   lhs_layout = "mk", rhs_layout = "kn", acc_type = "i32"}
  : vector<16x32xi8>, vector<32x16xi8>, vector<16x16xi32>
    -> vector<16x16xi32>
```

Pass Pipeline 위치

- canonicalize/cse 로 vector IR 를 단순화
- linalg-vectorize 이후 vector.contract 를 분석
- npu-vector-contract-tensorize 에서 native op 로 rewrite
- npu-vector-transfer-legalize 에서 transfer 를 DMA/SRAM path 로 정리
- npu-kernel-schedule 에서 barrier, double-buffering, command ordering 결정

Pipeline String Example

```
              mlir-opt input.mlir   --pass-pipeline='builtin.module(
func.func(
  canonicalize,cse,
  linalg-tile-to-vector-size,
  linalg-vectorize,
  canonicalize,cse,
  npu-vector-contract-tensorize,
  npu-vector-transfer-legalize,
  npu-kernel-schedule
))'
```

Testing with FileCheck

- Positive: supported vector.contract 가 npu_kernel.matmul 로 바뀌는지 확인
- Negative: unsupported shape/dtype/mask 에서는 tensorization 금지
- CHECK-SAME 으로 m/n/k/layout/acc_type attr 확인
- CHECK-NOT 으로 vector.contract 잔존 여부 확인
- diagnostic test 로 rejection reason 이 유지되는지 확인

FileCheck Template

```
            // RUN: mlir-opt %s --npu-vector-contract-tensorize | FileCheck %s
// CHECK-LABEL: func.func @m16n16k32_i8
// CHECK: npu_kernel.matmul
// CHECK-SAME: m = 16
// CHECK-SAME: n = 16
// CHECK-SAME: k = 32
// CHECK-NOT: vector.contract

// CHECK-LABEL: func.func @unsupported_k48
// CHECK-NOT: npu_kernel.matmul
// CHECK: vector.contract
```

Debug Recipe

- Print IR before/after vectorization and tensorization passes
- Log rejection reason through notifyMatchFailure or diagnostic attr
- Count tensorized ops with pass statistics
- Dump vector shapes and indexing maps in debug mode
- Correlate tensorized op count with runtime perf counter

Performance Traps

- Unsupported edge mask 가 silent fallback 으로 이어지는 경우
- layout transpose/repack 비용이 matmul 이득보다 큰 경우
- K split 이 accumulator spill 을 유발하는 경우
- transfer_read permutation 이 banking conflict 를 만드는 경우
- requant/activation epilogue 가 별도 kernel 로 분리되어 bandwidth 가 증가하는 경우

HW Architect Notes

- native tile shape 는 datapath width, SRAM banking, accumulator file size 와 함께 결정
- compiler-visible ISA contract 에는 dtype, layout, mask, saturation/rounding 을 명시
- M/N/K tile 한 가지로 모든 model 을 커버하기 어렵기 때문에 multiple tile shapes 필요
- vector.contract mapping 이 잘 되도록 weight layout 과 offline packing format 을 설계

Exercise A - transfer_read Annotation

- base memref 와 memory space 표시
- indices 가 tile origin 과 어떻게 연결되는지 표시
- vector shape 를 M/K 또는 K/N 으로 해석
- permutation_map 과 in_bounds/mask/padding 을 audit

Exercise B - Contract Mapping

- indexing_maps 에서 M/N/K dimension 찾기
- iterator_types 에서 reduction dimension 확인
- lhs/rhs/acc/result type 을 target tile spec 과 비교
- NPU matmul attr m/n/k/layout/acc_type 작성

Exercise C - Legality Matrix

- M16N16K32 INT8 target 을 기준으로 tensorization 가능 여부 판단
- K48, transposed RHS, BF16, masked edge tile 사례를 각각 분류
- repair 가능하면 split/transpose/repack/pad/predication 중 하나 선택
- repair 불가능하면 negative diagnostic 문구 작성

Homework

- BF16 vector.contract 예제를 작성하고 F32 accumulator policy 를 명시
- M32N8K64 native tile target description YAML 작성
- edge tile M=13,N=16,K=32 처리 정책 선택 및 trade-off 설명
- npu-vector-contract-tensorize pass negative tests 3 개 작성

Takeaways

- Vector Dialect 는 NPU tile compute 를 표현하기 좋은 target-neutral IR 입니다 .
- vector.contract 는 tensorization 의 중심이지만 , shape/dtype/layout/mask legality 없이는 위험합니다 .
- transfer_read/write 는 tile movement 와 boundary semantics 를 담으므로 DMA lowering 전에 audit 해야 합니다 .
- 좋은 NPU compiler 는 tensorization success/failure 를 테스트와 diagnostic 으로 명확히 남깁니다 .

References

- MLIR Vector Dialect official documentation
- MLIR Passes: convert-vector-to-LLVM, convert-vector-to-SCF, target-specific lowering
- MLIR Linalg Dialect and vectorization lowering path
- MLIR Transform Dialect for fine-grained transformation orchestration